Student Management System

Java Object-Oriented Programming Project Report

Prepared by:

- Names:Elmenoufy Mohamed Ezat Abdelhamid/Rashid Mohamed Omer
- Students ID:202401010544/202501010714

Course:
Object-Oriented Programming (Java)

Lecturer:
Dr. Nazmirul Izzad Bin Nasir

University:
City University Malaysia

Submission Date:27/07/2026

Table of Contents

---

1. Introduction

The Student Management System is a console-based Java application developed using Object-Oriented Programming (OOP) principles. The system is designed to manage student information efficiently through a simple menu-driven interface.

The application allows users to perform essential student management operations such as adding new students, displaying all students, searching for a student by ID, updating student information, deleting student records, and counting the total number of students.

The project demonstrates the implementation of Java programming concepts including classes, objects, inheritance, abstraction, encapsulation, polymorphism, collections, loops, conditional statements, and user input handling.

---

2. Project Objectives

The objectives of this project are:

- Develop a Java console application.
- Apply Object-Oriented Programming principles.
- Manage student records efficiently.
- Improve programming and problem-solving skills.
- Demonstrate the use of Java Collections Framework.
- Build a user-friendly menu-driven system.

---

3. Problem Description

Managing student information manually can lead to data redundancy and errors. Educational institutions require a simple system that allows users to store and manage student records efficiently.

This project solves the problem by providing a console-based application where users can:

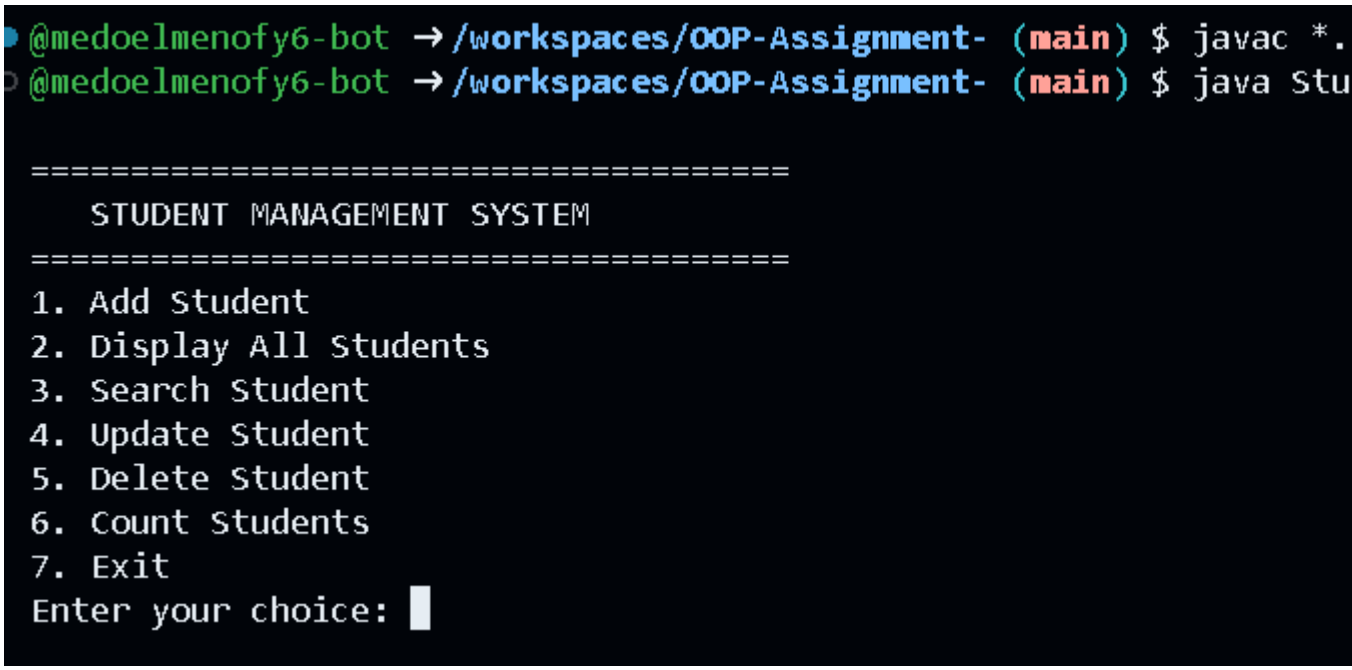
- Add new students
- Search student records
- Update existing information
- Delete records
- Display all students
- Count total students

The system stores all student records using Java's ArrayList collection.

---

4. System Features

The application provides the following functionalities:

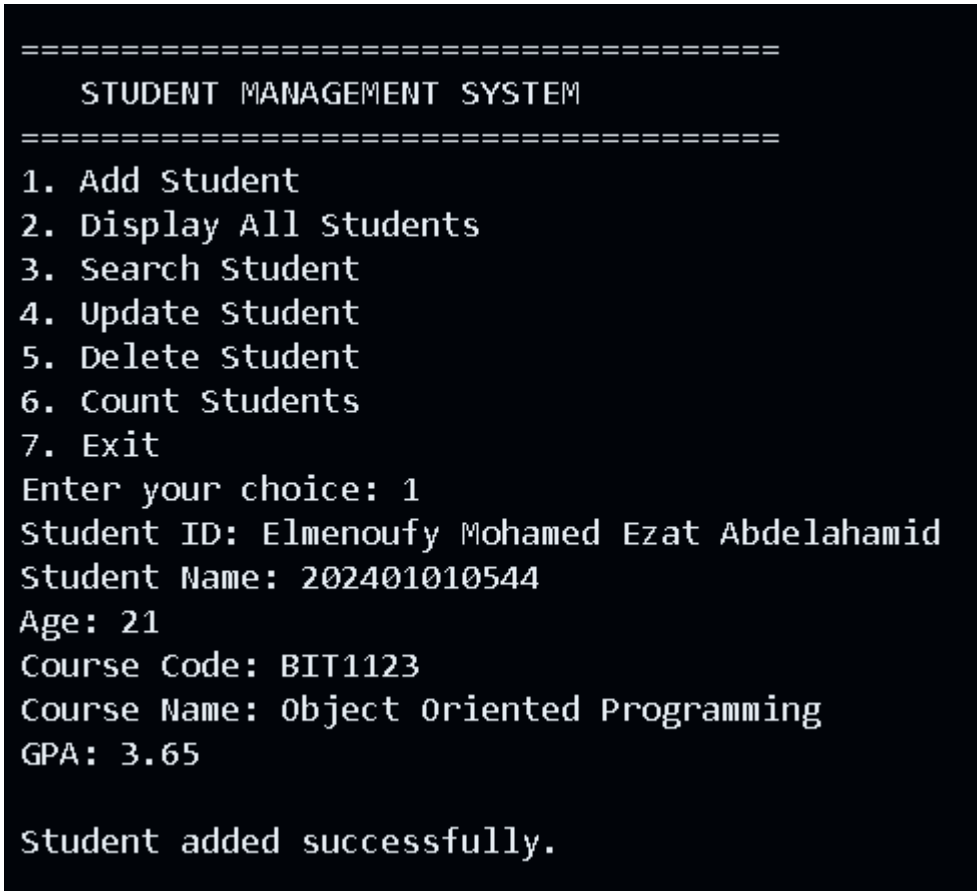


Add Student

Users can add a new student by entering:

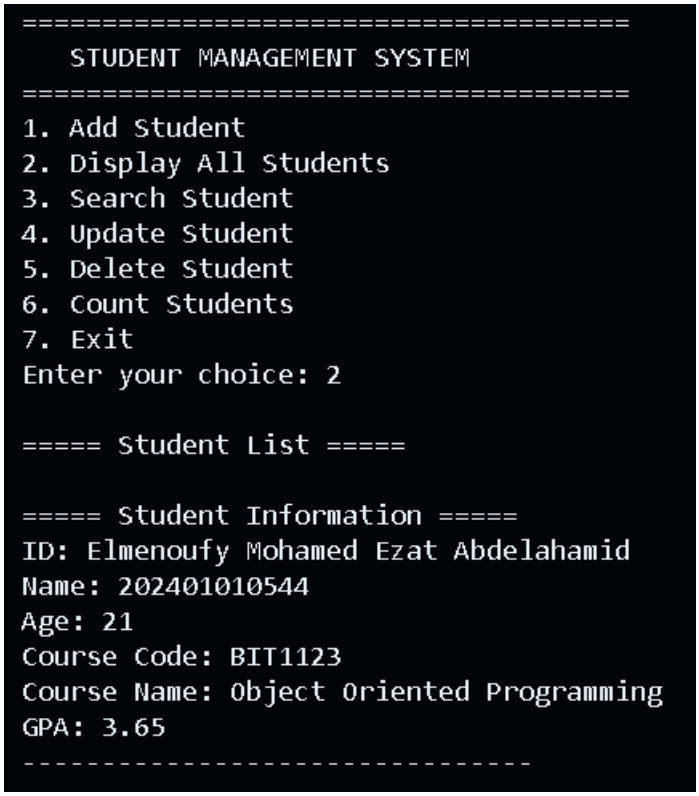
- Student ID
- Student Name
- Age
- Course Code
- Course Name
- GPA

The program validates the Student ID to prevent duplicate entries.

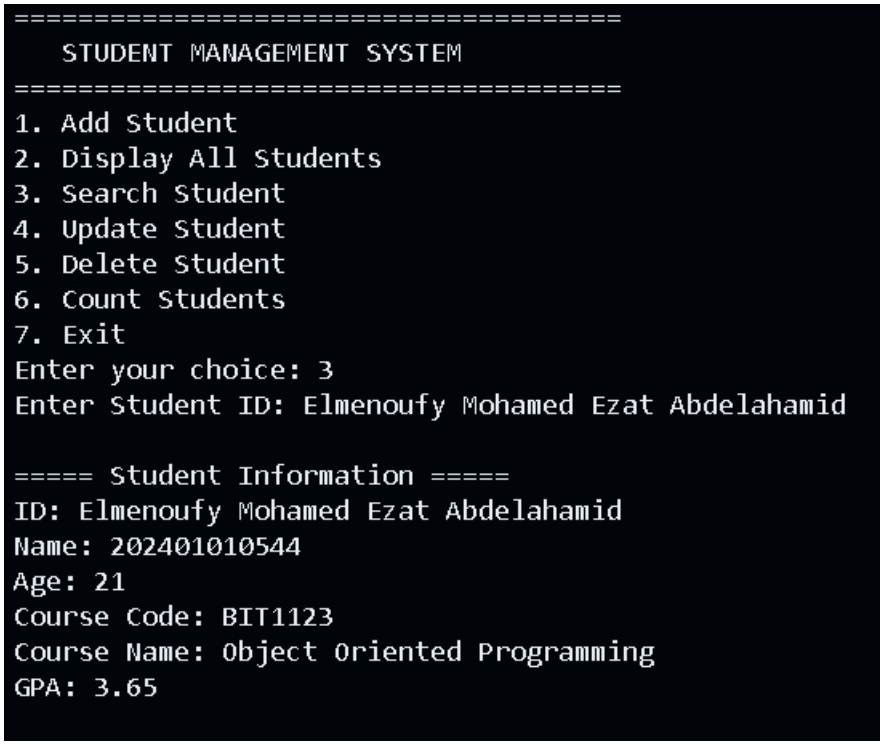


Display All Students

Displays all saved students together with their complete information.



Search Student



Allows users to search for a student using the Student ID.

Update Student

Users can modify:

- Name
- Age
- Course
- GPA

```
=====================================
   STUDENT MANAGEMENT SYSTEM
=====================================
1. Add Student
2. Display All Students
3. Search Student
4. Update Student
5. Delete Student
6. Count Students
7. Exit
Enter your choice: 4
Enter Student ID to Update: Elmenoufy Mohamed Ezat Abdelahamid
New Name: Elmenoufy Mohamed Ezat Abdelahamid
New Age: 21
New Course Code: BIT1123
New Course Name: Object Oriented Programming
New GPA: 3.8
Student updated successfully.
```

Delete Student

Deletes a student record using the Student ID.

```
=====================================
   STUDENT MANAGEMENT SYSTEM
=====================================
1. Add Student
2. Display All Students
3. Search Student
4. Update Student
5. Delete Student
6. Count Students
7. Exit
Enter your choice: 5
Enter Student ID to Delete: Elmenoufy Mohamed Ezat Abdelahamid
Student deleted successfully.
```

Count Students

Displays the total number of registered students.

```
=====================================
   STUDENT MANAGEMENT SYSTEM
=====================================
1. Add Student
2. Display All Students
3. Search Student
4. Update Student
5. Delete Student
6. Count Students
7. Exit
Enter your choice: 6
Total Students = 0
```

---

5. UML Class Diagram

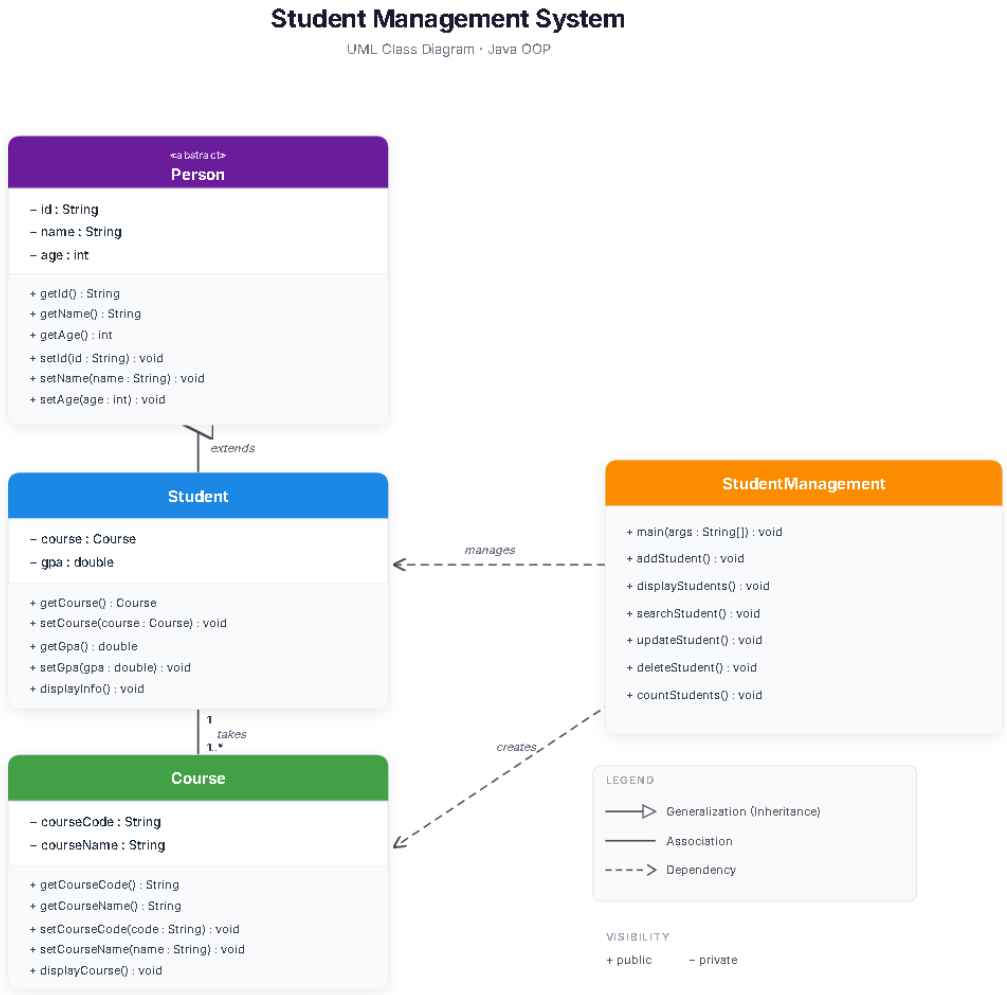


Figure 1: UML Class Diagram of the Student Management System.

---

6. Object-Oriented Programming Concepts

6.1 Class

The project contains four main classes:

- Person
- Student
- Course
- StudentManagement

Each class represents a specific responsibility within the system.

6.2 Object

Objects are created whenever a new student or course is added.

Example:

```
Course course = new Course(code, courseName);Person student = new Student(id, name, age, course, gpa);
```

6.3 Encapsulation

Private attributes are accessed using getter and setter methods, ensuring controlled access to data.

Example:

- getId()
- setId()
- getGpa()
- setGpa()

6.4 Inheritance

The Student class inherits from the abstract Person class.

```
Person
   ↑
Student
```

This allows Student to reuse the common attributes and methods defined in Person.

6.5 Abstraction

Person is implemented as an abstract class containing common properties and an abstract method named `displayInfo()`.

The Student class provides its own implementation of this method.

6.6 Polymorphism

The project uses polymorphism through the following statement:

```
Person student = new Student(...);
```

The overridden `displayInfo()` method is called according to the actual object type.

6.7 ArrayList

The application stores student objects inside an `ArrayList<Person>`.

This enables dynamic storage and easy management of student records.

6.8 Scanner

The Scanner class is used to receive user input from the keyboard.

6.9 Control Structures

The application makes use of:

- if statements
- switch statements
- for loops
- do-while loop

These structures control the program flow and user interaction.

---

7. Program Screenshots

Insert screenshots of the following:

- Main Menu
- Add Student
- Display Students
- Search Student
- Update Student
- Delete Student
- Count Students

---

8. Conclusion

This project successfully demonstrates the application of Object-Oriented Programming principles in Java. The Student Management System provides an efficient way to manage student records through a simple console interface.

The implementation of inheritance, encapsulation, abstraction, and polymorphism improves code organization, maintainability, and reusability. Additionally, the use of Java collections allows dynamic data management without relying on a database.

Overall, this project enhanced understanding of Java programming concepts and provided practical experience in developing real-world console applications.

---

9. References

1. Oracle. *Java Documentation*. https://docs.oracle.com/javase/
2. Herbert Schildt. *Java: The Complete Reference*. McGraw-Hill.
3. Oracle Java Tutorials. https://docs.oracle.com/javase/tutorial/

10. github link

https://github.com/medoelmenofy6-bot/OOP-Assignment-/tree/main